

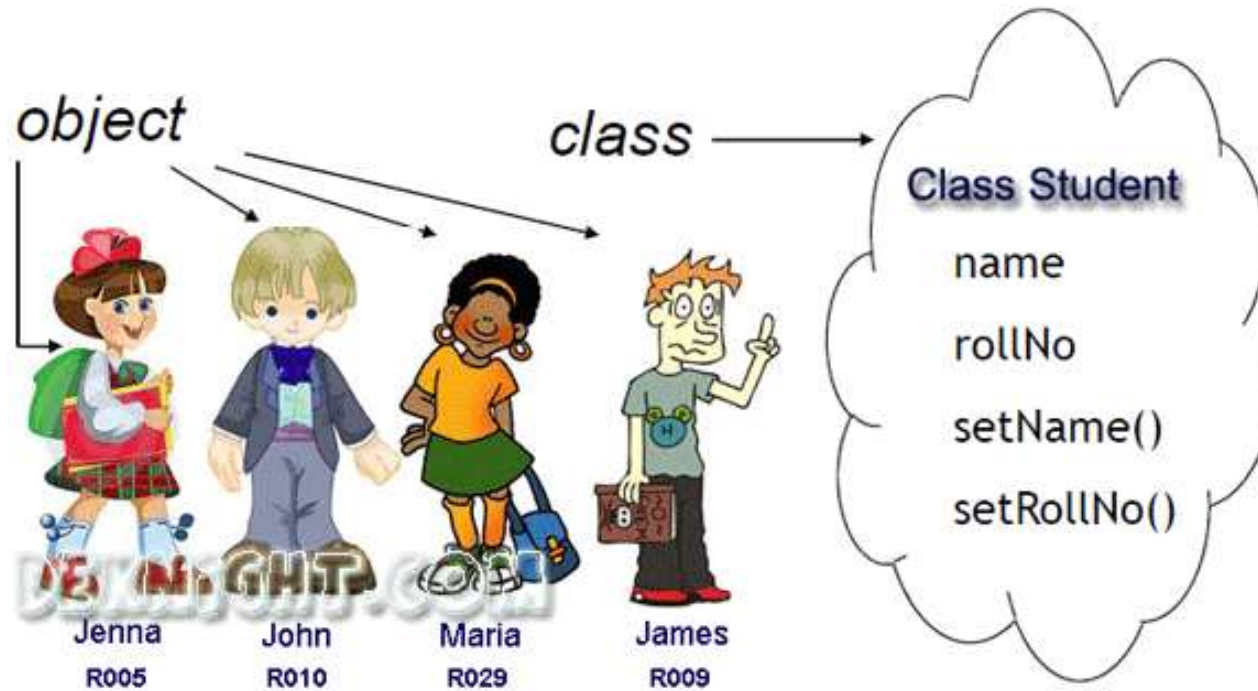
Bài 2:

Lớp và đối tượng

Lớp trong java

- Class là tập hợp các thuộc tính (*properties*) cùng với những phương thức (*methods*) đặc trưng cho một loại đối tượng nào đó cần dùng đến trong chương trình
- Trong lập trình hướng đối tượng, người ta ví class như là những “khuôn mẫu” dùng để “đúc” ra những đối tượng (*objects*) cho chương trình

Lớp trong java



Khai báo lớp

class <<ClassName>>

{

<<type>> <<field1>>;

...

<<type>> <<fieldN>>;

Khai báo các trường

Khai báo các phương thức

<<type>> <<method1>>([parameters]) {

// body of method

}

...

<<type>> <<methodN>>([parameters]) {

// body of method

}

}

Khai báo lớp

```
public class Employee{  
    public String fullname;  
    public double salary;  
  
    public void input(){  
        Scanner scanner = new Scanner(System.in);  
        System.out.print(" >> Full Name: ");  
        this.fullname = scanner.nextLine();  
  
        System.out.print(" >> Salary: ");  
        this.salary = scanner.nextDouble();  
    }  
  
    public void output(){  
        System.out.println(this.fullname);  
        System.out.println(this.salary);  
    }  
}
```

Trường



Phương thức

Lớp Employee có 2 thuộc tính là fullname và salary và 2 phương thức là input() và output()

Tạo & sử dụng đối tượng

Tạo đối tượng của 1 class đã được định nghĩa

Cú pháp

<className> <object> = new <className>();

staff1 là object của class Employee

```
public static void main(String[] args) {
```

```
    Employee staff1 = new Employee();|
```

```
}
```

Tạo & sử dụng đối tượng

Truy xuất đến thuộc tính/phương thức của đối tượng

Cú pháp

<object>.<thuocTinh>;

<object>.<phuongThuc>();

Tạo & sử dụng đối tượng

```
31 public static void main(String[] args) {  
32  
33     Employee staff1 = new Employee();  
34     staff1.input();  
35     staff1.output();  
36 }
```

Chú ý:

- ☐ Từ khóa **new** được sử dụng để tạo mới đối tượng trong java
- ☐ Biến staff1 chứa tham chiếu đến đối tượng
- ☐ Sử dụng dấu (.) để truy xuất đến các thành phần public bên trong đối tượng

Hàm khởi tạo - Constructor

- ❑ Constructor là gì ?
- ❑ Tại sao phải có constructor
- ❑ Khi nào nên sử dụng constructor
- ❑ Có bao nhiêu loại constructor
- ❑ Constructor được gọi khi nào ?

Hàm khởi tạo - Constructor

Lưu ý:

- Tên của phương thức Constructor phải giống với tên lớp
- Constructor luôn là public và không có kiểu trả về
- Có thể có nhiều phương thức constructor với các tham số khác nhau

```
10 public Employee(){  
11  
12 }  
13  
14 public Employee(String fName, double Salary){  
15     this.fullName = fName;  
16     this.salary = Salary;  
17 }
```

Overloading method

Trong một lớp, ta có thể định nghĩa nhiều phương thức có cùng tên với nhau, nhưng các tham số của các phương thức này phải khác nhau (Khác kiểu, khác số lượng, thứ tự tham số)

Overloading method

VD: nạp chồng hàm **sum**

```
3 public class Calculator {  
4  
5  
6     public int sum(int x, int y)  
7     {  
8         return (x + y);  
9     }  
10  
11  
12     public int sum(int x, int y, int z)  
13     {  
14         return (x + y + z);  
15     }  
16  
17  
18     public double sum(double x, double y)  
19     {  
20         return (x + y);  
21     }  
22 }
```

Từ khóa **this**

- Đại diện cho đối tượng hiện tại của class
- Sử dụng để tham chiếu đến các thành viên của class một cách tường minh, tránh bị nhầm lẫn với các biến khác.

```
5 public class Employee {  
6  
7     public String fullName;  
8     public double salary;  
9  
10    public Employee(){  
11  
12    }  
13  
14    public Employee(String fName, double Salary){  
15        this.fullName = fName;  
16        this.salary = Salary;  
17    }
```

Modifier: xác định phạm vi truy xuất

Modifier dùng để xác định phạm vi truy cập các thuộc tính, phương thức bên trong lớp

private: chỉ được phép truy xuất nội bộ trong class

public: truy xuất mọi nơi (công khai)

protected: chỉ truy xuất được trong lớp hiện tại và lớp kế thừa

Default:

- là public với các lớp cùng nằm trong package
- là private với các lớp khác package

Đóng gói và che dấu thông tin trong class

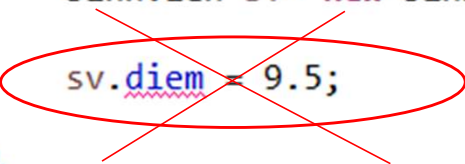
- ❑ **Encapsulation** là đặc tính quan trọng trong lập trình hướng đối tượng.
- ❑ **Encapsulation** cho phép che giấu thông tin, giúp bảo vệ dữ liệu thành viên, tránh những can thiệp không mong muốn từ bên ngoài class.

Đóng gói và che dấu thông tin trong class

Để che giấu thông tin, ta sử dụng từ khóa `private` cho các field là dữ liệu thành viên của lớp.

```
3 public class SinhVien {  
4  
5     private String HoTen;  
6     private double diem;  
7  
8 }
```

```
5 public static void main(String[] args) {  
6  
7     SinhVien sv= new SinhVien();  
8  
9     sv.diem = 9.5;  
10  
11 }
```



Đóng gói và che dấu thông tin trong class

Để truy cập vào các thuộc tính được che dấu của đối tượng ta phải dùng các phương thức setter/getter

```
14 public double getDiem() {  
15     return diem;  
16 }  
17 public void setDiem(double diem) {  
18     this.diem = diem;  
19 }
```

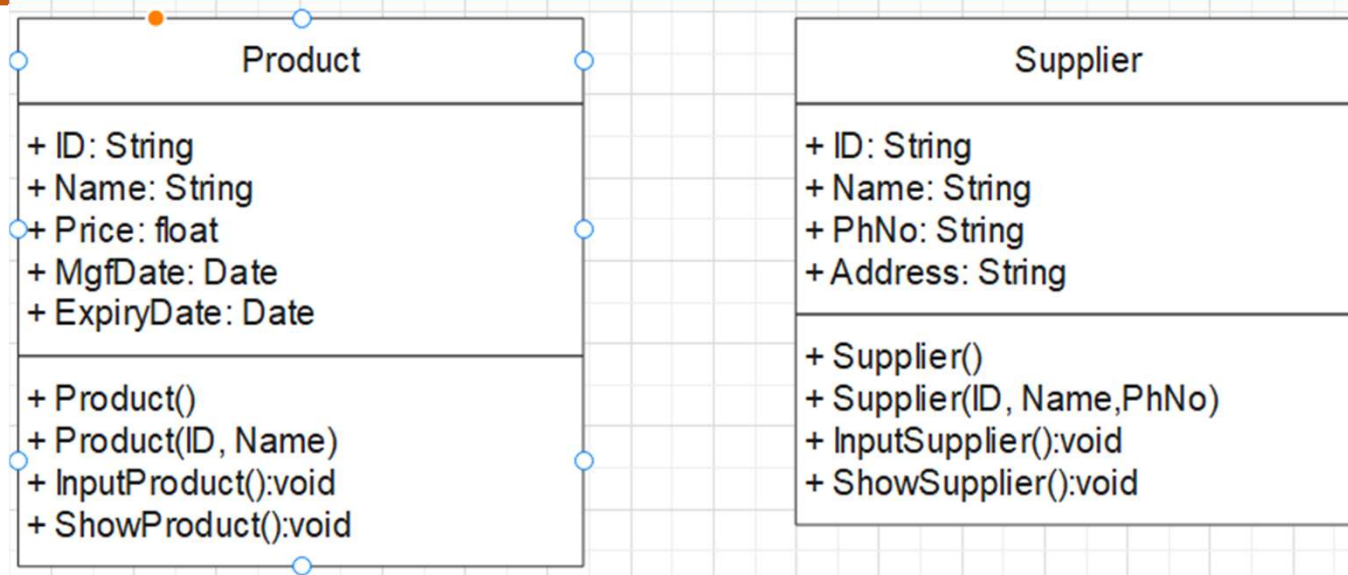
```
6  
7     SinhVien sv= new SinhVien();  
8  
9     sv.setDiem(9.5);  
10  
11 }
```

Ví dụ: Get/Set thuộc tính của đối tượng

```
public class SinhVien{
    private String hoTen;
    private double diem;
    public void setHoTen(String hoTen){
        this.hoTen = hoTen;
    }
    public String getHoTen(){
        return this.hoTen;
    }
    public void setDiem(double diem){
        if(diem < 0 || > 10){
            System.out.println("Điểm không hợp lệ");
        }
        else{
            this.diem = diem;
        }
    }
    public String getDiem(){
        return this.diem;
    }
}
```

```
public class MyClass{
    public static void main(String[] args){
        SinhVien sv = new SinhVien();
        sv.setHoTen("Nguyễn Văn Tèo");
        sv.setDiem(20);
    }
}
```

Bài tập áp dụng



Yêu cầu:

1. Cài đặt lớp Product (Sản phẩm) , Supplier (Nhà cung cấp)
2. Che dấu thuộc tính và xây dựng các phương getter, setter để truy cập các thuộc tính.
3. Viết chương trình cho phép nhập và in ra danh sách sản phẩm, danh sách Nhà cung cấp.